

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

# Import OpenCV for image processing algorithms
# Import Numpy for mathematical matrix operations
# Import Matplotlib to visualize the output on screen

# Create a simple 100x100 black road image
bg = np.zeros((100, 100, 3), dtype=np.uint8)

# Draw straight lane lines (Ground Truth)
cv2.line(bg, (30, 100), (30, 0), (255, 255, 255), 2)
cv2.line(bg, (70, 100), (70, 0), (255, 255, 255), 2)

# =====
# SYSTEM FAILURE: What the camera actually sees in bad conditions
# =====
# 1. Geometric Distortion: Wide lenses warp the straight lanes into a curve
camera_matrix = np.array([[100, 0, 50], [0, 100, 50], [0, 0, 1]], dtype=np.float32)
dist_coeffs = np.array([-0.5, 0, 0, 0], dtype=np.float32) # Negative value warps it inward
warped_frame = cv2.undistort(bg, camera_matrix, dist_coeffs * -1)

# 2. Radiometric Loss: Blinding white fog layers cover up the lane data
foggy_frame = cv2.addWeighted(warped_frame, 0.4, np.ones_like(bg) * 200, 0.6, 0)

# =====
# SYSTEM RESTORATION: Reversing the physics models to clean the data
# =====
# 1. Fix Lighting (Radiometric Restoration): Strip out the fog layer
def remove_fog(img):
    # Reverse blending: pull the lane contrast back out of the white fog
    return cv2.convertScaleAbs(img, alpha=2.5, beta=-150)

clear_frame = remove_fog(foggy_frame)

# 2. Fix Shapes (Geometric Restoration): Straighten the warped lane lines back out
final_clean_feed = cv2.undistort(clear_frame, camera_matrix, dist_coeffs)

# =====
# VISUALIZATION: Plotting the results
# =====
fig, axes = plt.subplots(1, 2, figsize=(10, 5))

axes[0].imshow(foggy_frame)
axes[0].set_title('Corrupted Feed\n(Curved & Obscured Lines)')
axes[0].axis('off')

axes[1].imshow(final_clean_feed)
axes[1].set_title('Restored AI Feed\n(Lines Straightened & Clear)')
axes[1].axis('off')

plt.show()

```

Corrupted Feed
(Curved & Obscured Lines)



Restored AI Feed
(Lines Straightened & Clear)

